

RAG Course Assistant — Project README

IIT Bombay CSE elective-selection assistant. Combines semantic search over syllabus content with SQL-based grade analytics, routed automatically by an LLM via native tool calling — built specifically to demonstrate the judgment that **structured data (grades) should be queried, not embedded**, while **unstructured data (syllabi) should be searched semantically, not filtered**.

Status: data layer complete and validated. Architecture fully specified. Implementation not yet started.

1. Data layer

`courses` table — 91 rows

Scraped from `https://www.cse.iitb.ac.in/academics/courses` (an Angular/MDB single-page app — required Playwright, not plain HTML scraping) plus one manually added course (CS738, found via grade data that had no matching course record).

Column	Notes
<code>course_code</code>	Primary key, e.g. <code>CS725</code>
<code>title</code> , <code>department</code>	
<code>home_page</code>	Course's own page, where one exists
<code>autumn_instructor</code> , <code>spring_instructor</code>	
<code>semester</code>	<code>'Autumn'</code> , <code>'Spring'</code> , or <code>'Autumn, Spring'</code> (only CS101 is both)
<code>credits</code> , <code>prerequisites</code> , <code>duration</code> , <code>course_type</code>	
<code>syllabus_text</code>	The field that gets embedded for semantic search
<code>references_text</code>	Textbook citations — metadata, not embedded
<code>source_url</code> , <code>last_updated</code>	
<code>level</code>	<code>'UG'</code> or <code>'PG'</code> — derived from the course code's numeric part (below 500 = UG, 500+ = PG). Verified against all 91 real codes with zero parse failures. 31 UG / 60 PG.

`grade_distributions` table — 582 rows across 81 courses

Manually collected from ASC (behind IITB login, so scraping wasn't possible) using a purpose-built paste-and-parse tool (`grade-entry-tool.html`).

Column	Notes
<code>course_code</code>	FK → <code>courses.course_code</code>
<code>grade</code>	Letter grade, e.g. <code>AA</code> , <code>BC</code> , <code>FR</code>
<code>student_count</code>	

No `academic_year` or `semester` column — deliberately dropped since all data is from a single year (2025) and most courses run in only one semester per year; the rare exception (CS101) just has its grades summed together under one course code, which is fine for "how tough is this course overall."

10 courses have no grade data (checked — not a rename/replace situation, confirmed by scanning syllabus text for "replaces"/"reused" language, found nothing):

- Autumn: `CS416M`, `CS616`, `CS694`, `CS6010`, `CS6012`, `CS6013`, `CS7001`, `CS721`
- Spring: `CS6009`, `CS713`

CS694 is a Seminar course, almost certainly pass/fail and never letter-graded — not really "missing," just not applicable. The other 9 are unresolved; likely either didn't run that year or weren't graded via ASC. Deprioritized rather than chased further.

`grade_points` table — verified against IITB's official grading rules

Built after initially getting one grade wrong (`AP` was assumed to mean "Audit Pass" — actually a full 10-point grade, same tier as `AA`; audit is `AU`, a separate ungraded grade). Verified via web search against IITB's actual rules document before use.

grade	points	is_pass
AP, AA	10	1
AB	9	1
BB	8	1
BC	7	1
CC	6	1
CD	5	1
DD	4	1 (minimum passing grade)
FF, FR, DX	0	0
II, AU, W, DR	NULL	NULL (excluded from averages entirely)
PP	NULL	1
NP	NULL	0

Any future "easiest course" / "average grade" query should JOIN against this table rather than the LLM guessing a mapping.

2. Tools built along the way

File	Purpose
<code>scrape_playwright.py</code>	Scrapes the CSE courses SPA — expands semester accordions, clicks each course's modal, parses fields by their literal UI labels
<code>course-entry-tool.html</code>	Paste a course's modal text (or any similarly-labeled syllabus page) → auto-parses fields → editable form → exports SQL. In-memory only (no persistent storage — that API doesn't work outside Claude's own artifact preview)
<code>grade-entry-tool.html</code>	Paste an ASC grade-stats page → auto-detects course code and grade/count pairs → editable rows → exports SQL. Handles the "first grade block vs. secondary section breakdown" ambiguity correctly (confirmed against real CS728/CS101 samples)
<code>schema.sql</code>	Base schema: <code>courses</code> + <code>grade_distributions</code>
<code>schema_migration_v2.sql</code>	Adds <code>level</code> column + <code>grade_points</code> table

All course and grade INSERT batches were validated by actually executing them against a real SQLite engine (not eyeballed) — catching real bugs like unescaped apostrophes, missing column values, and foreign-key mismatches before they reached the working database.

3. Final architecture

User ⇄ Streamlit UI (multi-turn conversation)



LLM (Claude API, tool calling)



search_syllabi
(Chroma, vector)



query_database
(SQLite, text-to-SQL)

Router

No custom classifier. Native LLM tool/function calling — the model itself decides which tool(s) to call based on the two tool descriptions it's given. Can call one tool, both, or one after seeing the other's result, all within one conversational turn.

`search_syllabi`

- **Store:** Chroma (separate from SQLite)
- **Embedding model:** sentence-transformers, local — no API key, no cost, runs on-device
- **Granularity:** one vector per course, not sub-document chunks (syllabi are short enough that a course is already the right retrieval unit — chunking would just fragment single-course hits)
- **What's embedded:** `(title + syllabus_text)` concatenated (title alone can carry keywords the body never repeats)
- **Missing-data fallback:** when `(syllabus_text = 'N/A')`, embed the title alone rather than skipping the course

`query_database`

- **Approach:** freeform text-to-SQL — the LLM writes its own `(SELECT)`, not a fixed menu of pre-built functions
- **Guardrails:** read-only connection, reject non-SELECT statements, reject statement-stacking (`(;` followed by more content), enforce a row-count cap when no `(LIMIT)`/course filter is present
- **Full schema (both tables + grade_points) is embedded directly in the tool's description string**, so the LLM always has it — no separate schema-discovery step

Interface

Streamlit, multi-turn. Conversation history is passed to the LLM each call, so follow-up questions work without extra engineering — this falls out of the tool-calling architecture for free.

4. Router specification (tool schemas, system prompt, error handling)

Full detail in `(router_spec.md)`. Summary of the key routing rule, arrived at by stress-testing the design against realistic "first two weeks of semester" student questions:

If the student names an exact course code or title, prefer `(query_database)` over `(search_syllabi)` — even for topic questions like "what does CS725 cover" — since a direct `(WHERE course_code = ...)` lookup is more reliable than semantic search once you already know the target. `(search_syllabi)` is reserved for "help me find a course about X" when the course itself is unknown.

Also encoded in the system prompt: mandatory use of `(grade_points)` for any grade-average calculation, and two honest, stated limitations the assistant should surface rather than guess around:

- **No class-timetable data exists.** Timing/clash questions can't be answered.

- **Grade data is missing for ~10 courses.** If a query returns nothing, say so — don't invent numbers.
-

5. Known limitations (accepted scope boundaries, not bugs)

- No lecture timetable data — timing/clash questions are out of scope
 - `prerequisites` text is inconsistently formatted across courses (`"CS 213"` vs `"CS213"` vs `"CS101 & CS213 M"`), so prerequisite-matching queries may miss variants
 - 10 courses have no grade data (listed above)
 - Grade averages from very-low-enrollment courses (e.g. 2-3 students) can be misleading — system prompt tells the LLM to flag this explicitly rather than present a "10.0 average" as meaningful on its own
-

6. Not yet built

- Embedding script (courses table → Chroma index)
- Tool-execution wrappers implementing the error-handling spec
- The actual Claude API tool-use loop
- Streamlit UI shell

Architecture is fully decided for all of the above — remaining work is implementation, not further design.